

GPU 101

How a machine for pictures learned to compute $A + B$

Minjae Gwon

POSTECH

September 8, 2026

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

- Rendering and shaders
- GPGPU and CUDA

3. The execution engine

- Warps and SIMT
- SMs and GPU organization

4. Following A + B

- From tensors to threads
- Scheduling and memory
- Utilization and completion

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

3. The execution engine

4. Following A + B

What should make more sense after this talk?

- **The GPU you observe**

What are VRAM and all those execution resources?

- **The frameworks you use**

What sits below a PyTorch or Hugging Face operation?

- **The platform NVIDIA built**

How did graphics hardware become a general compute platform?

Pictures → Execution engines → $C = A + B$

One image. Very different ways to paint it.



Parallel work becomes easy to see when the output is a picture.

CPU vs. GPU

One paintball at a time.

Many paintballs together.

► [Watch the demo](#)

NVIDIA NVISION 08 / MythBusters demo · video re-upload: HOSTKEY

1. Motivation

2. From pictures to programs

- Rendering and shaders
 - GPGPU and CUDA
-

3. The execution engine

4. Following A + B

The original deadline: draw the next frame

3D rendering

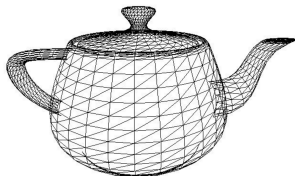


Image credit: Henrik Wann Jensen

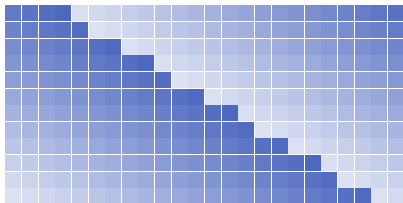
Input: description of a scene:

3D surface geometry (e.g., triangle mesh)
surface materials, lights, camera, etc.

Output: image of the scene

Geometry + materials + lights + camera → an image

Millions of positions. The same kind of calculation.



Different positions, different inputs

1920 × 1080

output pixels per frame

× **60 frames / second**

≈ **124 million**

output pixels per second

The opportunity is repeated work across independent data.

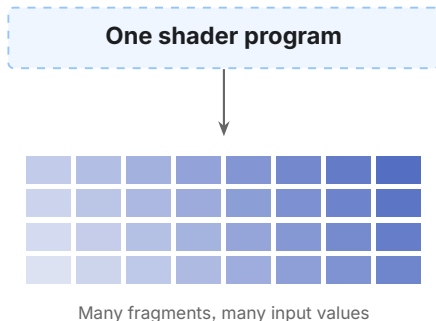
A shader is a program for a graphics stage

ONE FRAGMENT'S WORK

```
base    = read_surface_color(uv)
light   = max(dot(normal, light_dir), 0)
output  = base * light
```

- Programmable behavior replaces a fixed formula.
- A fragment is a candidate contribution to a pixel.

Write how one surface point is shaded; apply it across the surface.



Do those numbers have to be colors?

THE GRAPHICS INTERPRETATION



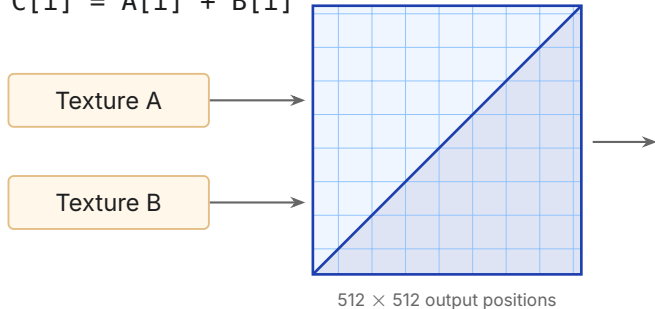
THE COMPUTE INTERPRETATION



GPGPU: General-Purpose Computing on GPUs

To add two arrays, first draw two triangles

$$C[i] = A[i] + B[i]$$



- 1 Set the output image size.
- 2 Cover it with two triangles.
- 3 Add values in a fragment shader.

Read the image back as an array.

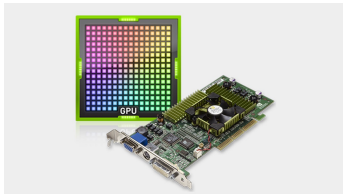
A draw call to the GPU; an array operation to the researcher.

“All this just to do $A + B$ ”



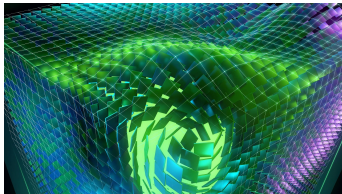
- The entry point still speaks graphics.
- Fragment programs have restricted output locations.
- Sharing data between executions is constrained.

CUDA made compute a first-class interface



1999

GeForce 256



2006

CUDA architecture

`launch()`

CUDA Toolkit 1.0

2007

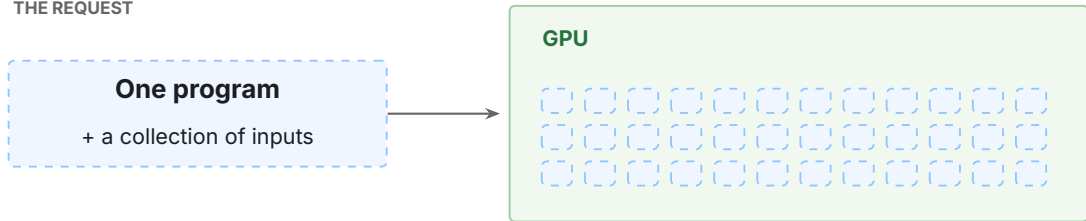
CUDA Toolkit 1.0

Allocate buffers. Run a kernel. Read and write data directly.

Images / 2006: NVIDIA corporate timeline · Toolkit 1.0: NVIDIA guide, June 2007

“Run this program many times.”

THE REQUEST



The data-parallel structure survives.

Many logical executions

Now look inside the machine that accepts that request.

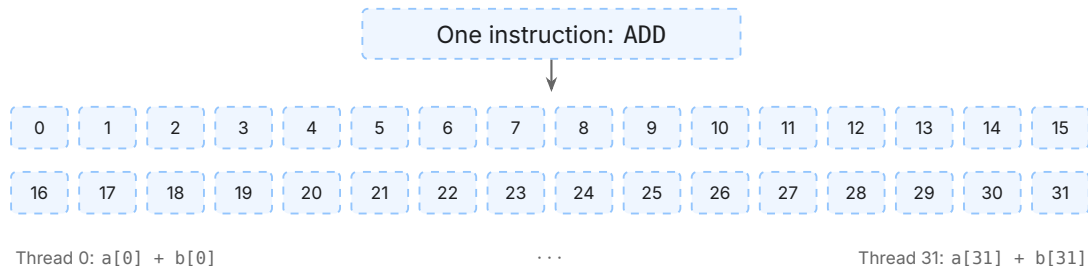
2. From pictures to programs

3. The execution engine

- Warps and SIMT
 - SMs and GPU organization
-

4. Following A + B

A warp groups 32 logical threads

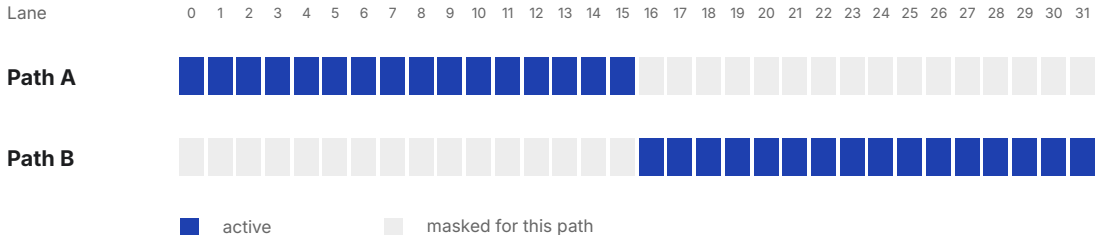


SIMT Single Instruction, Multiple Threads

A warp describes grouped execution; it is not a 32-core component.

Same program does not always mean the same path

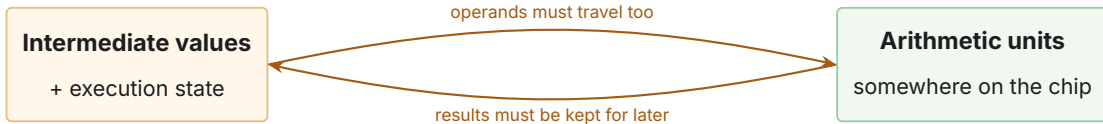
```
if (lane < 16) { path_A(); } else { path_B(); }
```



Divergent paths are processed with different active threads.

Arithmetic needs state, and state has to live somewhere

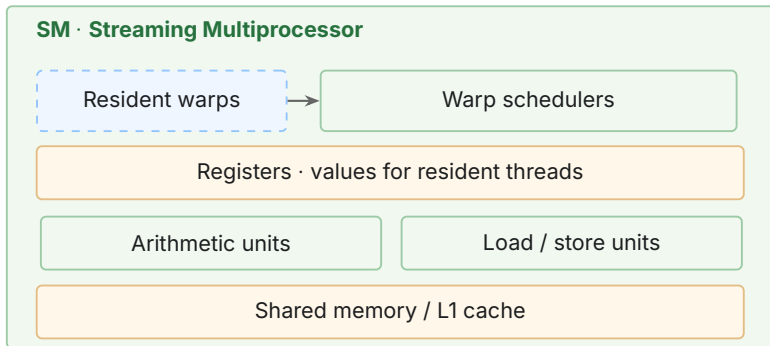
A DESIGN THOUGHT EXPERIMENT



Store values. Track progress. Choose the next instruction.

Adding execution units also creates a data-movement problem.

An SM keeps execution and its working state close



Manage many warps.

Keep their state nearby.

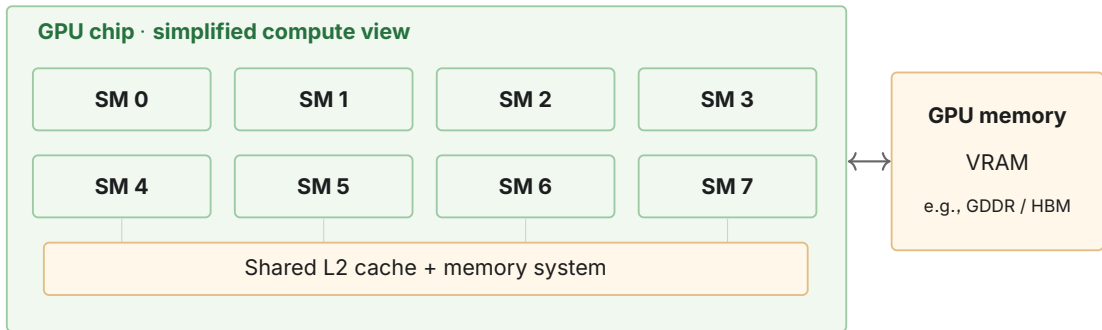
Feed ready instructions to execution units.

Conceptual organization.
Internal partitions vary by architecture.

Warp = grouped work. SM = the hardware that executes it.

The execution engine

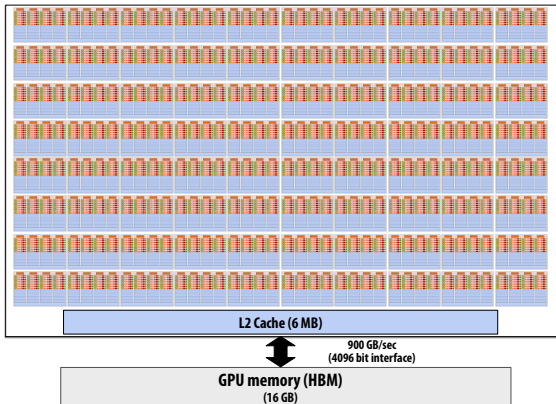
Scale the GPU by repeating that execution engine



Share control within a warp; keep execution resources local to an SM.

NVIDIA CUDA Programming Guide · GPU Hardware Model · illustrative SM count

A concrete example: the NVIDIA V100



80 SMs

Repeated execution engines

Shared L2 cache

A cache shared by the SMs

HBM2 memory

Stores the inputs and outputs

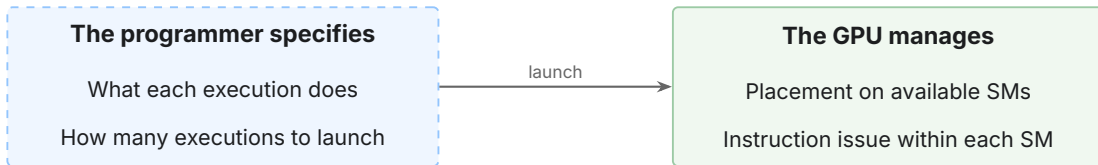
Historical example, not a universal GPU specification.

Stanford CS149, Fall 2025 · slide 52 · V100 configuration shown in the lecture

4. Following A + B

- From tensors to threads
- Scheduling and memory
- Utilization and completion

Describe the work; the GPU places it



PyTorch chooses CUDA implementations underneath tensor operations.

A work description is not a physical placement plan.

Start with one line of PyTorch

```
import torch  
A = torch.randn(1024, 1024, device="cuda")  
B = torch.randn(1024, 1024, device="cuda")  
C = A + B
```

A



B



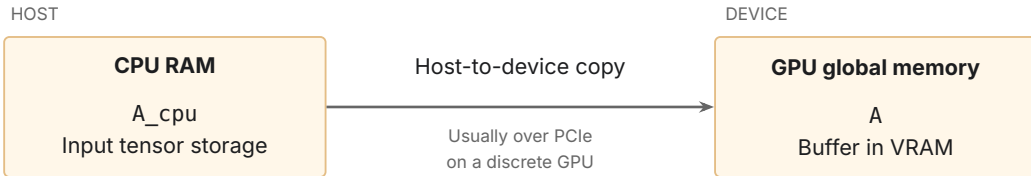
=

C



Each output needs only its own two inputs. No neighbor must finish first.

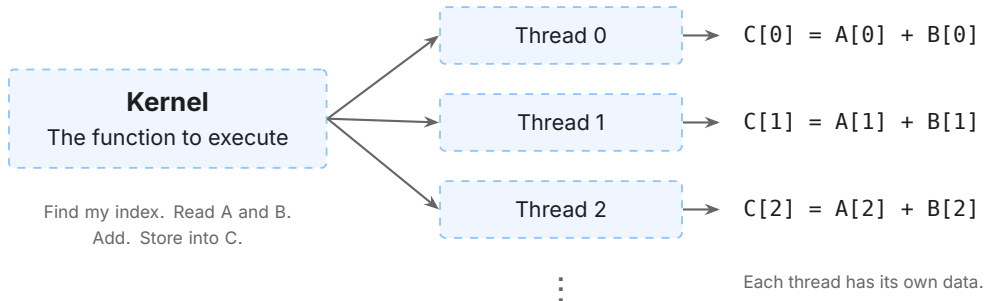
If the input starts in CPU memory, copy it to the GPU



```
A_cpu = torch.randn(1024, 1024, device="cpu")  
A = A_cpu.to("cuda")
```

With `device="cuda"`, our original example already creates GPU tensors.

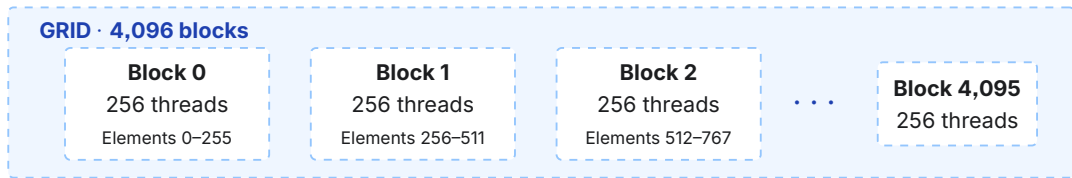
One program, many logical executions



A kernel is code. A thread is one logical execution of that code.

How many executions do we launch?

$1024 \times 1024 = \mathbf{1,048,576}$ elements



```
add<<<4096, 256>>>(device_a, device_b, device_c, 1024 * 1024);
```

4,096 blocks $\times$ 256 threads = 1,048,576 logical threads.

Every thread can find its own element

```
__global__ void add(  
    const float* a,  
    const float* b,  
    float* c,  
    int element_count  
) {  
    int index = blockIdx.x * blockDim.x  
               + threadIdx.x;  
    if (index < element_count) {  
        c[index] = a[index] + b[index];  
    }  
}
```

BLOCK 3, THREAD 10

$$\underbrace{3 \times 256}_{\text{block offset}} + \underbrace{10}_{\text{local index}}$$

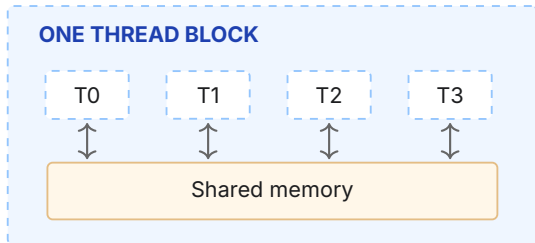
778

C[778] = A[778] + B[778]

The bounds check protects the final block when the size is not divisible by 256.

Same code, different built-in IDs, different data.

A block is also a cooperation boundary



Share data locally

Threads in a block can exchange data.

Synchronize within the block

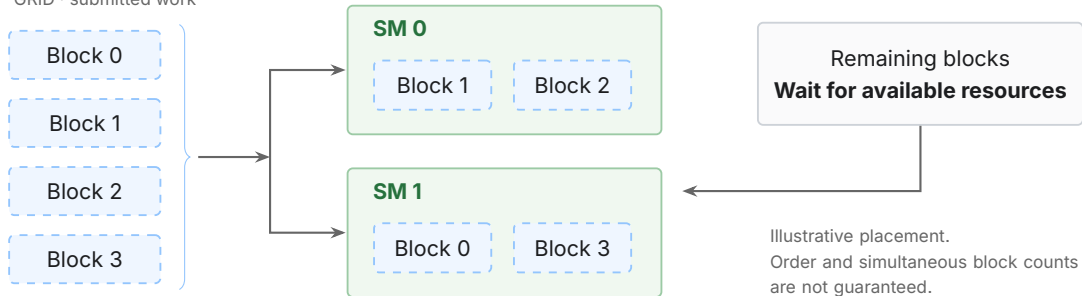
`__syncthreads()` provides a block-wide barrier.

256 is our choice. A common device limit is 1,024 threads per block; kernel resources can constrain it further.

This addition needs no shared intermediate values or block barrier.

Many blocks share a finite set of SMs

GRID · submitted work



One block runs on one SM. An SM may host several blocks.

A 256-thread block contains eight warps

BLOCK 0 · 256 threads

Warp 0  0–31

Warp 1  32–63

Warp 2  64–95

Warp 3  96–127

Warp 4  128–159

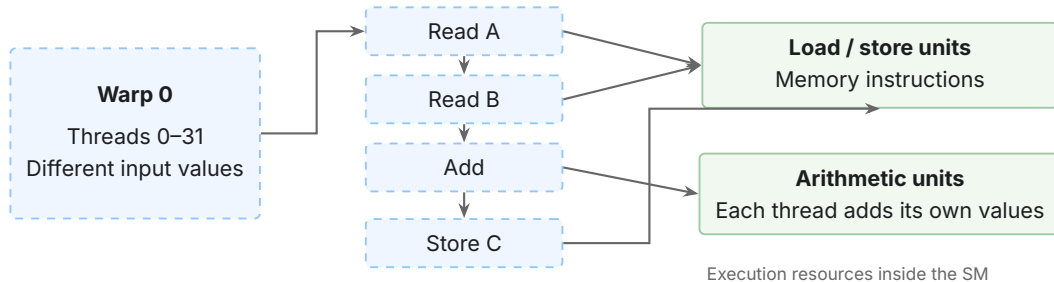
Warp 5  160–191

Warp 6  192–223

Warp 7  224–255

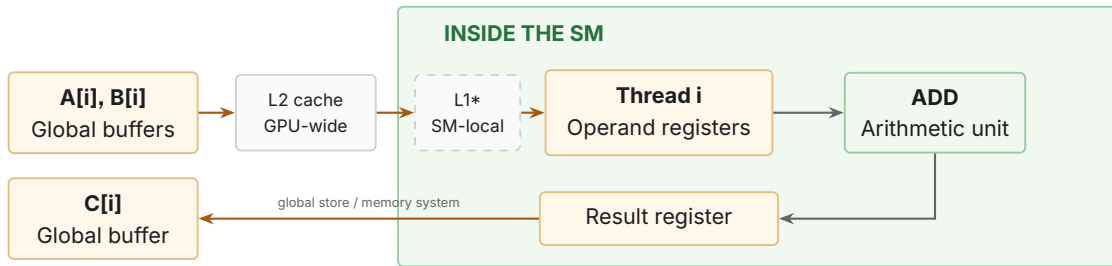
Within a block, consecutive groups of 32 threads form warps.

A warp needs different units for different instructions



The instruction is shared; the data belongs to each thread.

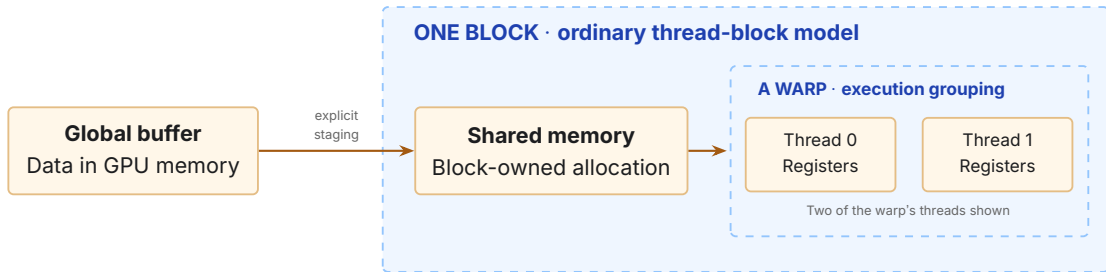
For this addition, operands are loaded into thread registers



*Caches may serve the load; L1 use and the load/store path depend on architecture and policy.

The add kernel uses per-thread registers. Shared-memory staging is unnecessary.

Shared memory is explicit storage for block cooperation

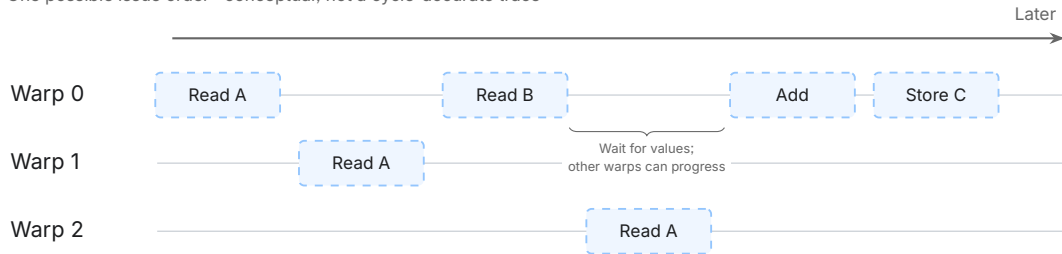


Useful when threads reuse or exchange data. This addition does not take this optional path.

There is no separate “warp memory” tier between shared memory and registers.

The scheduler issues the next ready instruction

One possible issue order · conceptual, not a cycle-accurate trace



Registers retain each thread's values between instructions.

A warp does not have to finish its whole program before another runs.

Warp width and execution width are different numbers

VOLTA / V100 EXAMPLE

One warp
32 threads



One SM partition



A 32-thread FP32 arithmetic instruction is processed over **two cycles**.

V100 has four warp schedulers / partitions per SM. Other instructions and architectures differ.

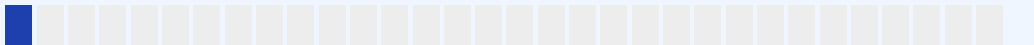
32 threads does not imply 32 dedicated units finishing in one clock.

What if we launch just one thread?

```
add<<<1, 1>>>(device_a, device_b, device_c, 1);
```

ONE SM

BLOCK 0 · WARP 0



Lane 0: one valid thread

Lanes 1–31: unused

One active thread in a warp on one SM.

More threads in one block still means one SM

ONE LARGE BLOCK

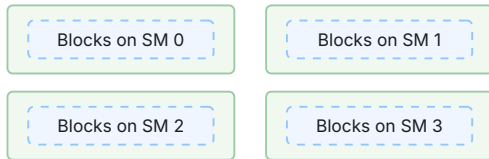
`add<<<1, 1024>>>(...)`



If supported by the device and kernel

MANY BLOCKS

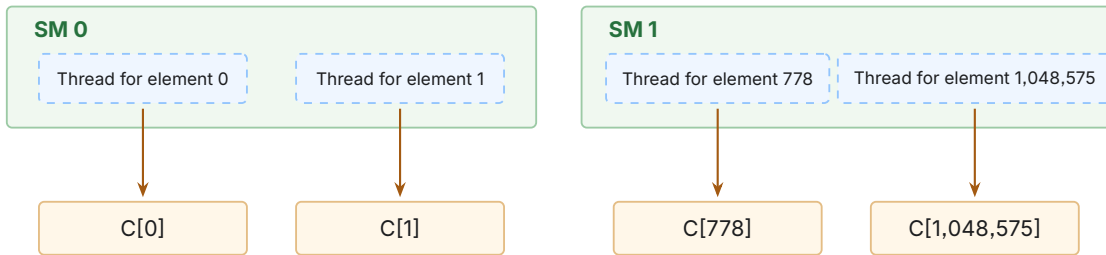
`add<<<4096, 256>>>(...)`



Blocks available for multiple SMs

Fill the warps, and provide blocks that can spread across SMs.

The result is already in C, in GPU memory

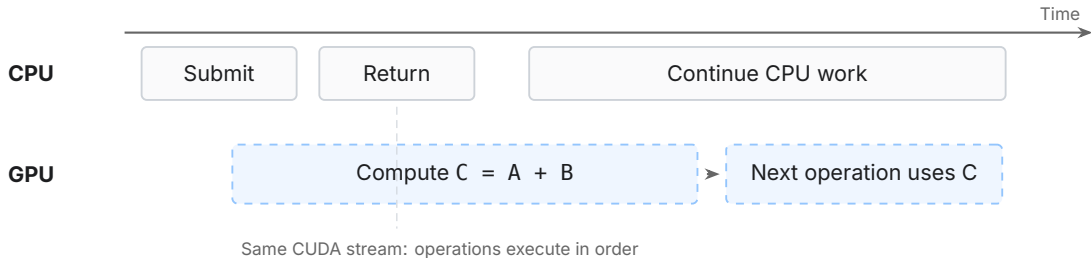


Distinct output addresses in one result buffer

Selected elements shown; placement is illustrative

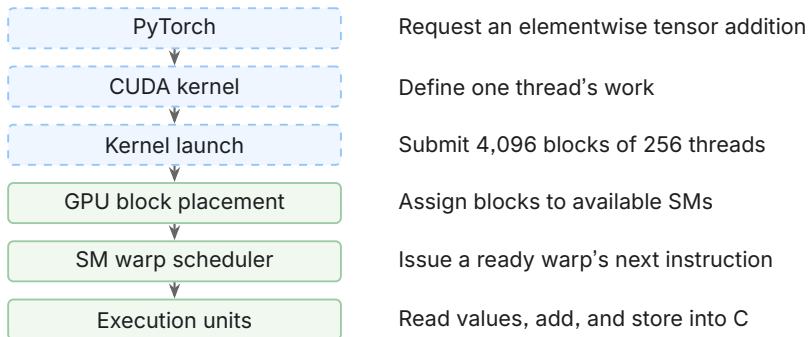
Each thread stores its own result. The CPU does not reassemble the matrix.

Python can return before the GPU finishes



- GPU operations are normally submitted asynchronously.
- Reading results on the CPU or measuring GPU time needs appropriate synchronization.

Follow the request all the way down



One tensor operation becomes many scheduled instructions on real hardware.

Keep the work and the machine separate

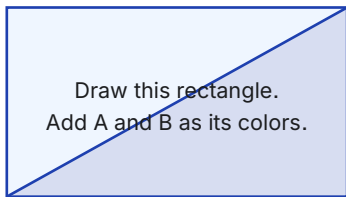
Kernel	The GPU program to execute
Thread	One logical execution of that program
Block	A group of threads that can cooperate
Grid	All blocks submitted for this kernel launch
Warp	A 32-lane execution group within one block
SM	The hardware engine that executes resident warps

Grid $\neq$ GPU. Block $\neq$ SM. Thread $\neq$ a dedicated core.

Following $A + B$

From two triangles to one line

THEN



NOW

$$C = A + B$$

The constant idea: apply the same computation to many inputs.

GPU 101

How a machine for pictures learned to compute $A + B$

Educational scope This introduction uses simplified kernels and conceptual diagrams. Hardware details, memory behavior, and launch configurations vary by architecture and software version; the examples do not specify PyTorch's actual implementation. NVIDIA and Stanford materials are credited for reference and do not imply endorsement. Interpretations and any errors are the presenter's responsibility.